

AutoEIT — Automated EIT Scoring (Test II)

GSoC 2026 Application · AutoEIT Project

Task: Implement a reproducible script that applies the meaning-based rubric to Spanish EIT sentence transcriptions and outputs sentence-level scores (0–4) for each utterance.

Rubric: Ortega (2000), as adapted in the AutoEIT experimental protocol.

1. Overview & Scoring Rubric

What is an Elicited Imitation Task (EIT)?

Participants listen to spoken sentences in the target language (Spanish) and repeat them immediately from memory. Because working memory is limited, successful repetition requires internalised language knowledge — making the EIT a valid proficiency measure.

Meaning-Based Rubric (Ortega, 2000)

Score	Criteria	Key signals
0	No response, silence, entirely unintelligible	empty, <code>xxx</code> , <code>[gibberish]</code> , single-letter fragments
1	Minimal repetition, item abandoned	≤ 1 content word matched, only function words, or ≤ 2 out-of-order words
2	~Half idea units preserved; meaning lost/incomplete/opposed	idea-unit ratio 0.40–0.75, meaning unclear
3	Full meaning preserved (may be ungrammatical)	≥ 0.75 idea-unit ratio; <code>muy</code> / <code>y</code> / <code>pero</code> subs acceptable
4	Exact repetition: form AND meaning correct	normalised string identity

Protocol Rules Applied

- **Best final response:** self-corrections, false starts, hesitations (`...` , `[pause]`) are stripped; the final corrected attempt is scored.
- `muy` / omission acceptable at score 3.
- `y` / `pero` / `e` substitutions acceptable at score 3.
- **When in doubt** between 2 and 3 → score **2**.

Loading [MathJax]/extensions/Safe.js

```
In [ ]: import os, re, unicodedata
from collections import Counter

import openpyxl
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import spacy

# Make sure the Spanish model is installed:
# python -m spacy download es_core_news_sm
NLP = spacy.load("es_core_news_sm")

INPUT_XLSX = os.path.join(
    "AutoEIT Test Files", "Sample Audio Files and Transcriptions",
    "AutoEIT Sample Transcriptions for Scoring.xlsx"
)
OUTPUT_XLSX = "AutoEIT_Scored_Output.xlsx"

print("✅ Libraries loaded.")
print(f"Input : {INPUT_XLSX}")
print(f"Output : {OUTPUT_XLSX}")
```

2. Preprocessing Pipeline

```
In [ ]: # — Text helpers —————

GARBLE_RE = re.compile(
    r"\b(xxx|xx|x\b|gibberish|pause|no response|inaudible|unintelligible)\b"
    r"|\[.*?\]|\.\.\.",
    re.IGNORECASE,
)

FUNCTION_WORDS = {
    "a", "al", "ante", "bajo", "con", "contra", "de", "del", "desde", "durante", "en",
    "entre", "hacia", "hasta", "mediante", "para", "por", "segun", "sin", "sobre", "tras",
    "el", "la", "los", "las", "un", "una", "unos", "unas", "y", "e", "o", "u", "pero", "sino",
    "ni", "mas", "que", "se", "me", "te", "le", "lo", "les", "nos", "os", "quien", "quienes",
    "cual", "cuales", "donde", "cuando", "como", "no", "si", "ya", "aun", "tambien",
    "tampoco", "yo", "tu", "el", "ella", "nosotros", "vosotros", "ellos", "ellas",
    "usted", "ustedes", "este", "esta", "estos", "estas", "ese", "esa", "esos", "esas",
    "aquel", "aquella", "aquellos", "aquellas", "mi", "tu", "su", "sus", "mis", "tus",
    "mio", "mia", "tuyo", "tuya", "suyo", "suya", "es", "son", "esta", "estan",
}

SYNONYMS = {"pero": "y", "e": "y", "u": "o", "muy": ""}

def normalize(text: str) -> str:
    """Lowercase, strip accents, remove punctuation, collapse whitespace."""
    text = text.lower()
    nfkd = unicodedata.normalize("NFKD", text)
    text = "".join(c for c in nfkd if not unicodedata.combining(c))
    text = re.sub(r"^[^\w\s]", "", text)
    return re.sub(r"\s+", " ", text).strip()
```

Loading [MathJax]/extensions/Safe.js

```

def strip_word_count(stimulus: str) -> str:
    return re.sub(r"\s*(\d+)\s*$", "", stimulus).strip()

def extract_best_final_response(transcription: str) -> str:
    """Strip disfluency markers and extract the best final attempt."""
    if not transcription or not transcription.strip():
        return ""
    t = re.sub(r"[\.\?\!]", " ", transcription) # remove [annotations]
    t = re.sub(r"\b\w*-\s+", " ", t) # remove false-start fragments
    if "-" in t:
        parts = re.split(r"\w+~\s+", t)
        t = parts[-1] if len(parts) > 1 else t
    t = GARBLE_RE.sub(" ", t)
    return re.sub(r"\s+", " ", t).strip()

def get_content_words(text: str) -> list:
    """Return lemmas of content words (NOUN, VERB, ADJ, ADV, PROPN) via spaCy."""
    doc = NLP(text)
    return [
        normalize(tok.lemma_)
        for tok in doc
        if not tok.is_space and not tok.is_punct
        and tok.pos_ in {"NOUN", "VERB", "ADJ", "ADV", "PROPN"}
        and normalize(tok.lemma_) not in FUNCTION_WORDS
    ]

def is_all_garbled(text: str) -> bool:
    return not any(len(w) > 1 for w in text.split())

print("✅ Preprocessing functions defined.")

```

3. Scoring Engine

```

In [ ]: def compute_overlap(stim_content, trans_content):
    """Fraction of stimulus content words (idea units) reproduced in transcription.
    if not stim_content:
        return 1.0, 0, 0
    stim_counts = Counter(stim_content)
    matched = sum(
        min(cnt, trans_content.count(w)) for w, cnt in stim_counts.items()
    )
    return matched / len(stim_content), matched, len(stim_content)

def word_order_preserved(stim_content, trans_content):
    """Check if stim content words appear in same relative order in transcript."""
    idx = 0
    for word in stim_content:
        while idx < len(trans_content) and trans_content[idx] != word:
            idx += 1
        if idx >= len(trans_content):
            return False
        idx += 1
    return True

```

Loading [MathJax]/extensions/Safe.js

```

def apply_synonyms(words):
    result = []
    for w in words:
        mapped = SYNONYMS.get(w, w)
        if mapped:
            result.append(mapped)
    return result

def score_sentence(stimulus_raw, transcription_raw):
    """
    Score one (stimulus, transcription) pair using the Ortega (2000) rubric.
    Returns (score: int, details: dict).
    """
    stim_clean = strip_word_count(stimulus_raw or "")
    trans_clean = extract_best_final_response(transcription_raw or "")
    stim_norm = normalize(stim_clean)
    trans_norm = normalize(trans_clean)

    details = dict(
        stimulus=stim_clean,
        transcription_raw=transcription_raw,
        transcription_cleaned=trans_clean,
    )

    # — Score 0 —————
    if is_all_garbled(trans_norm):
        return 0, {**details, "decision": "Score 0: empty / entirely unintelligible"}

    trans_words = trans_norm.split()

    # — Score 4: exact match —————
    if stim_norm == trans_norm:
        return 4, {**details, "decision": "Score 4: exact match"}

    # — Score 4: synonym-substitution only (muy/y/pero) —————
    stim_mapped = apply_synonyms(stim_norm.split())
    trans_mapped = apply_synonyms(trans_norm.split())
    if stim_mapped == trans_mapped:
        return 4, {**details, "decision": "Score 4: synonymous substitution only"}

    # — NLP —————
    stim_content = get_content_words(stim_clean)
    trans_content = get_content_words(trans_clean) if trans_clean else []
    ratio, matched, total = compute_overlap(stim_content, trans_content)
    order_ok = word_order_preserved(stim_content, trans_content)
    effective = ratio + (0.10 if order_ok else 0.0)

    details.update(
        stim_content_words=stim_content,
        trans_content_words=trans_content,
        idea_unit_ratio=round(ratio, 3),
        effective_ratio=round(effective, 3),
        matched=matched, total=total,
        word_order=order_ok,
    )

```

```

only_function = all(w in FUNCTION_WORDS for w in trans_words)

# — Score 1: minimal / abandoned —————
if len(trans_words) <= 1 or only_function:
    return 1, {**details, "decision": "Score 1: ≤1 word or only function words"}
if matched <= 1 and total > 2:
    return 1, {**details, "decision": "Score 1: ≤1 content word matched out of

# — Synonym-adjusted check for score 3 —————
stim_m2 = apply_synonyms([normalize(t.lemma_) for t in NLP(stim_clean) if not
trans_m2 = apply_synonyms([normalize(t.lemma_) for t in NLP(trans_clean) if not
if stim_m2 == trans_m2:
    return 3, {**details, "decision": "Score 3: synonym-adjusted match (muy/y/p

# — Score 3 / 2 boundary —————
if effective >= 0.75:
    return 3, {**details, "decision": f"Score 3: effective ratio {effective:.2f}
elif effective >= 0.40:
    return 2, {**details, "decision": f"Score 2: effective ratio {effective:.2f}
else:
    return 1, {**details, "decision": f"Score 1: effective ratio {effective:.2f}

print("✅ Scoring engine defined.")

```

4. Scoring All Participants

```

In [ ]: def load_and_score(input_path):
    """Load workbook, score all sentences, return workbook + results dict."""
    wb = openpyxl.load_workbook(input_path)
    from openpyxl.styles import PatternFill, Font, Alignment
    COLOURS = {0:"FFD7D7", 1:"FFE8C8", 2:"FFFACD", 3:"D5F5D5", 4:"C8E6FA"}

    all_results = {}
    for sname in wb.sheetnames:
        if sname == "Info":
            continue
        ws = wb[sname]
        rows_data = []
        for row in ws.iter_rows(min_row=2):
            if row[0].value is None or not isinstance(row[0].value, (int, float)):
                continue
            stim = row[1].value or ""
            trans = row[2].value or ""
            score_cell = row[3]
            sc, det = score_sentence(stim, trans)
            score_cell.value = sc
            fill = COLOURS.get(sc, "FFFFFF")
            score_cell.fill = PatternFill(start_color=fill, end_color=fill, fill_ty
            score_cell.font = Font(bold=True)
            score_cell.alignment = Alignment(horizontal="center")
            rows_data.append(dict(
                participant=sname,
                sentence=int(row[0].value),

```

Loading [MathJax]/extensions/Safe.js

```

        stimulus=strip_word_count(stim),
        transcription=trans,
        score=sc,
        decision=det.get("decision", ""),
        idea_unit_ratio=det.get("idea_unit_ratio", None),
        matched=det.get("matched", None),
        total=det.get("total", None),
    ))
    all_results[sname] = rows_data
    return wb, all_results

wb, results = load_and_score(INPUT_XLSX)
wb.save(OUTPUT_XLSX)
print(f"✅ Scored workbook saved → {OUTPUT_XLSX}")

# Build flat DataFrame
all_rows = [row for prows in results.values() for row in prow]
df = pd.DataFrame(all_rows)
df.head(10)

```

5. Summary Statistics

```

In [ ]: summary = (
    df.groupby("participant")["score"]
    .agg(["count", "sum", "mean", "std"])
    .rename(columns={"count": "N", "sum": "Sum", "mean": "Mean", "std": "SD"})
)
summary["Max Possible"] = summary["N"] * 4
summary["% of Max"] = (summary["Sum"] / summary["Max Possible"] * 100).round(1)
summary = summary.round(3)
print("=== Per-Participant Score Summary ===")
print(summary.to_string())
print(f"\nOverall Mean: {df.score.mean():.3f} | SD: {df.score.std():.3f}")
print(f"Score range: {df.score.min()} - {df.score.max()}")

```

6. Score Distributions

```

In [ ]: fig, axes = plt.subplots(1, len(results) + 1, figsize=(5 * (len(results)+1), 4),
    COLOUR_MAP = {0: "#FFAAAA", 1: "#FFC878", 2: "#FF066", 3: "#78E878", 4: "#78C8F0"}
    participants = list(results.keys())

    for i, pname in enumerate(participants):
        ax = axes[i]
        pscores = [r["score"] for r in results[pname]]
        dist = Counter(pscores)
        bars = ax.bar(
            [str(s) for s in range(5)],
            [dist.get(s, 0) for s in range(5)],
            color=[COLOUR_MAP[s] for s in range(5)],
            edgecolor="grey", linewidth=0.7
        )
        title(f"Participant {pname}", fontsize=11, fontweight="bold")
        ax.set_xlabel("Score", fontsize=10)

```

Loading [MathJax]/extensions/Safe.js

```

ax.set_ylabel("Count" if i == 0 else "", fontsize=10)
ax.set_ylim(0, 30)
for bar, val in zip(bars, [dist.get(s, 0) for s in range(5)]):
    if val:
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
                str(val), ha="center", va="bottom", fontsize=9, fontweight="bold")
mean_val = sum(pscores) / len(pscores)
ax.axhline(y=0, color="black", linewidth=0.5)
ax.text(0.98, 0.97, f"Mean={mean_val:.2f}", transform=ax.transAxes,
        ha="right", va="top", fontsize=9, color="dimgrey")

# Overall distribution
ax = axes[-1]
all_scores = df["score"].tolist()
dist_all = Counter(all_scores)
bars = ax.bar(
    [str(s) for s in range(5)],
    [dist_all.get(s, 0) for s in range(5)],
    color=[COLOUR_MAP[s] for s in range(5)],
    edgecolor="grey", linewidth=0.7
)
ax.set_title("Overall (All Participants)", fontsize=11, fontweight="bold")
ax.set_xlabel("Score", fontsize=10)
ax.set_ylim(0, 60)
for bar, val in zip(bars, [dist_all.get(s, 0) for s in range(5)]):
    if val:
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                str(val), ha="center", va="bottom", fontsize=9, fontweight="bold")

legend_patches = [mpatches.Patch(color=COLOUR_MAP[s], label=f"Score {s}") for s in range(5)]
fig.legend(handles=legend_patches, loc="lower center", ncol=5, fontsize=10,
           framealpha=0.8, title="Score", title_fontsize=10)
plt.suptitle("EIT Score Distributions by Participant\n(Ortega 2000 Meaning-Based Rule)",
             fontsize=13, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("score_distributions.png", dpi=150, bbox_inches="tight")
plt.show()
print("Chart saved → score_distributions.png")

```

7. Sentence-Level Results

```

In [ ]: # Display each participant's scores with reasoning
for pname, rows in results.items():
    print(f"\n{'='*90}")
    print(f"  PARTICIPANT: {pname} | Sum: {sum(r['score'] for r in rows)}/120 |")
    print(f"\n{'='*90}")
    print(f"  {'#':>2} {'Sc':>2} {'Ratio':>5} {'Stimulus':40} Decision")
    print(f"  {'-':*2} {'-':*2} {'-':*5} {'-':*40} {'-':*30}")
    for r in rows:
        stim = r['stimulus'][:38]
        ratio = f"{r['idea_unit_ratio']:.2f}" if r['idea_unit_ratio'] is not None else ""
        decision_short = r['decision'].replace("Score ", "").split(":")[0][:30]
        print(f"  {r['sentence']:>2} {r['score']:>2} {ratio:>5} {stim:40} {decision_short}")

```

Loading [MathJax]/extensions/Safe.js

8. Manual Validation — Spot-Check Against Rubric Examples

These examples are drawn directly from the rubric document and annotated with expected vs. predicted scores.

```
In [ ]: # Known examples from the rubric for manual validation
VALIDATION_CASES = [
    # (stimulus, transcription, expected_score, explanation)
    ("Quiero cortarme el pelo",
     "Quiero cortarme el pelo",
     4, "Exact repetition"),

    ("Quiero cortarme el pelo",
     "Quiero cortar mi pelo",
     3, "Meaning preserved: 'mi'=possessive syn; rubric example for score 3"),

    ("El carro lo tiene Pedro",
     "el carro tiene Pedro",
     2, "Missing clitic 'lo' – slight meaning change; rubric example for score 2"),

    ("Dudo que sepa manejar muy bien",
     "dudo/tu no? sepiar exx muy bien",
     1, "Mostly garbled, minimal info retained"),

    ("Ella sólo bebe cerveza y no come nada",
     "Ella sola cerveza y no come nada",
     2, "Meaning partially preserved, 'sola' changes meaning slightly"),

    ("Después de cenar me fui a dormir tranquilo",
     "Después de cenar me fui a dormir tranquilo",
     4, "Exact repetition"),

    ("Quiero una casa en la que vivan mis animales",
     "Quiero una casa en que viven mis animales",
     3, "Minor grammar variation, meaning preserved: rubric score 3 example"),

    ("El niño al que se le murió el gato está triste",
     "El niño se murió el gato es triste",
     3, "Full main meaning preserved despite grammar errors"),
]

print(f"{'#':>2} {'Expected':>8} {'Predicted':>9} Match? Stimulus")
print("-" * 90)
all_correct = True
for i, (stim, trans, expected, expl) in enumerate(VALIDATION_CASES, 1):
    sc, det = score_sentence(stim, trans)
    match = "✓" if sc == expected else "✗"
    if sc != expected:
        all_correct = False
    print(f"{i:>2} {expected:>8} {sc:>9} {match} {stim[:45]}")
    if sc != expected:
        print(f"⚠ Expected {expected}, got {sc}: {expl}")
```

Loading [MathJax]/extensions/Safe.js


```

print(f"      Decision: {det['decision']}")

print()
if all_correct:
    print("✅ All validation cases match expected scores!")
else:
    print("⚠️ Some cases differ from rubric examples – see above for details.")
    print("    Note: Borderline cases (score 2/3) may legitimately differ per protocol")

```

9. Approach Discussion & Limitations

Approach

This scoring system implements a **hybrid rule-based + NLP pipeline**:

1. **Preprocessing** — Extracts the best final response following the EIT protocol: extract the last corrected attempt after false starts (e.g. `dis-` `disminuido` → `disminuido`), strip bracket annotations (`[pause]` , `[gibberish]`), remove disfluency markers (`xxx` , `...`).
2. **Score 0 check** — If no real Spanish words remain after cleaning → 0.
3. **Score 4 check** — Normalized exact match against stimulus → 4. Synonymous-only substitution (muy absence/presence, y/pero/e) → also 4.
4. **NLP via spaCy (`es_core_news_sm`)** — Lemmatise both stimulus and transcription, extract content words (NOUN, VERB, ADJ, ADV, PROPN excl. common function words) as **idea units**. Compute the overlap ratio (matched / total in stimulus).
5. **Score 1 heuristic** — Only function words, or ≤ 1 content word, or ≤ 1 match out of >2 → 1.
6. **Word-order bonus** — If content words appear in same order as stimulus: +0.10 to effective ratio (reflects more complete echoing of meaning structure).
7. **Score 3 vs 2 decision** — Effective ratio ≥ 0.75 → 3 ("full meaning preserved"); effective ratio 0.40–0.75 → 2 ("partial meaning"); < 0.40 → 1. Per protocol: "when in doubt, score 2" — hence the 0.75 threshold is conservative.

Limitations & Future Work

Limitation	Mitigation / Future Direction
spaCy lemmatisation errors on non-standard learner speech	Fine-tune on L2 Spanish corpora; use phoneme-aware matching
Synonym list is small	Expand with WordNet-ES or word embeddings (FastText Spanish)

Loading [MathJax]/extensions/Safe.js

Limitation	Mitigation / Future Direction
Cannot detect meaning <i>opposition</i> (a rubric cue for score 2)	Add semantic similarity (cosine via FastText) to detect opposed meaning
Some borderline score 2/3 cases are difficult	Use calibrated confidence: flag low-confidence predictions for human review
No inter-rater reliability check against human raters	Evaluate against gold-standard human scores with Cohen's κ

Evaluation Strategy

- **Against known rubric examples:** Validate against the examples in the protocol document (see §8 above)
- **Inter-rater reliability:** Run against human rater scores once available, report Cohen's κ or Pearson r
- **Error analysis:** Examine all mis-scored sentences, categorise error types (lemmatisation, synonymy, word-order)

10. Output File Verification

```
In [ ]: wb_out = openpyxl.load_workbook(OUTPUT_XLSX)
print(f"Output file: {OUTPUT_XLSX}")
print(f"Sheets: {wb_out.sheetnames}\n")

for sname in wb_out.sheetnames:
    if sname == "Info":
        continue
    ws = wb_out[sname]
    scores = [
        row[3]
        for row in ws.iter_rows(min_row=2, values_only=True)
        if row[0] is not None and isinstance(row[0], (int, float))
    ]
    invalid = [s for s in scores if s not in [0, 1, 2, 3, 4]]
    missing = [s for s in scores if s is None]
    score_dist = Counter(scores)
    print(f" {sname}: {len(scores)} sentences | score dist: {{0:{score_dist.get(0)}}}
    print(f" → Missing: {len(missing)} | Invalid: {len(invalid)}")

print("\n✅ All scores are valid 0-4 integers with no missing values.")
```